Integration Testing II: Databases, Services, Doubles, and Automation

SE 413/513 Software Testing · Demo: demos/integration_testing

Dr. Kamrul Hasan, Grand Valley State University

Learning objectives

1

Databases

Test CRUD, constraints, and transactions against a real database, and check committed state through a second connection.

2

Services and doubles

Test an external dependency's failures with a fake over real HTTP, and tell a stub that checks from one that lies.

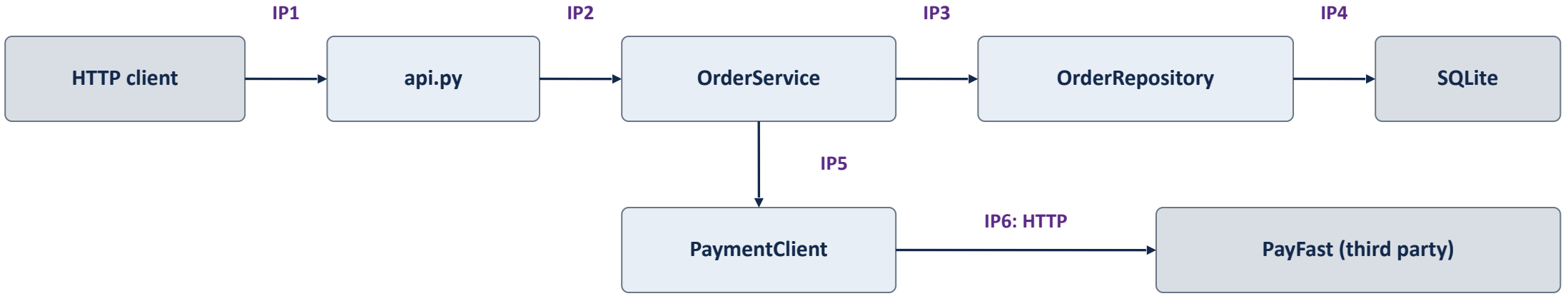
3

Automate

Organize integration tests with a folder and a marker, keep them fast and isolated, and run them in CI.

Reading: Week 9, Sections 8-13. **Demo folder:** `demo-testing-suite/demos/integration_testing`

Where we left off



`SHOP_PAYMENTS=v1`

The client sends "12.50" dollars; PayFast v2 wants 1250 cents.

Found in Part 1, at IP6

`SHOP_SERVICE=buggy`

Stock and order committed in two separate transactions.

Today, at IP3 + IP4

Which database should a test use?

Option	Good for	Watch out for
Mock of the repository	unit tests of the caller	no SQL, constraints, or transactions
`sqlite3 ":memory:"`	fast SQL and constraint tests	one connection only
file in tmp_path	all of the above, plus a second connection	a little slower
Production engine in a container	PostgreSQL/MySQL behavior	slower; Docker in CI

Read what was committed

```
@pytest.fixture
def stock(db_path):
    def read(sku):
        fresh = db.connect(db_path)
        try:
            return fresh.execute(
              "SELECT stock FROM products"
              " WHERE sku = ?", (sku,)
            ).fetchone()["stock"]
        finally:
            fresh.close()
    return read
```

Isolation: every test gets its own database, from the application's own schema, seeded with exactly the rows it needs.

The repository, against real SQLite

LIVE DEMO 1
8 passed, real SQLite

```
pytest tests/test_repository.py -v
```

```
test_schema_has_both_tables                      PASSED
test_create_schema_is_safe_to_run_again          PASSED
test_order_create_read_update_delete             PASSED
test_reserve_stock_refuses_to_oversell           PASSED
test_database_rejects_negative_stock             PASSED
test_database_rejects_an_order_for_an_un...      PASSED
test_uncommitted_changes_are_invisible_to...     PASSED
test_rollback_undoes_every_statement_in_...      PASSED
8 passed in 0.09s
```

The test is the driver

Bottom-up: tests call the repository the way OrderService eventually will.

What a mock can't check

Column defaults, generated ids, and SQL against the real schema.

Transactions

Uncommitted changes are invisible to another connection; a rollback undoes every statement.

Remove one line the review flagged

LIVE DEMO 2
R10: foreign keys

shop/db.py

```
def connect(path):
    connection = sqlite3.connect(path)
    connection.row_factory = sqlite3.Row
    connection.execute("PRAGMA foreign_keys = ON")
    return connection
```

tests/test_repository.py

```
def test_database_rejects_an_order_for_an_
        unknown_product(connection, repository):
    with pytest.raises(sqlite3.IntegrityError,
            match="FOREIGN KEY constraint failed"):
        with connection:
            repository.create_order(
                "NOPE", 1, 100, "ana@example.com")
```

Delete the line, run, restore

```
sed -i.bak '/PRAGMA foreign_keys/d' shop/db.py
pytest tests/test_repository.py
mv shop/db.py.bak shop/db.py
```

```
FAILED ...rejects_an_order_for_an_unknown_product
1 failed, 7 passed
```

A review finding, made permanent. SQLite ignores REFERENCES unless each connection enables foreign keys. Without this test, a harmless-looking cleanup would silently let orders point at products that do not exist.

One transaction, or two?

shop/service.py: one transaction

```
with self.connection:
    if not self.repository.reserve_stock(
            sku, quantity):
        raise OutOfStock(sku)
    order_id = self.repository.create_order(
        sku, quantity, total_cents,
        customer_email)
```

shop/service_buggy.py: two

```
with self.connection:  # commits the stock...
    if not self.repository.reserve_stock(
            sku, quantity):
        raise OutOfStock(sku)
with self.connection:  # ...before this can fail
    order_id = self.repository.create_order(
        sku, quantity, total_cents,
        customer_email)
```

What if the order insert fails? customer_email is NOT NULL. With one transaction, the stock reservation rolls back too. With two, the stock is already committed, for an order that does not exist.

Only one level catches it

LIVE DEMO 3
assert 3 == 5

```
SHOP_SERVICE=buggy pytest tests/test_order_service.py
```

```
in test_order_the_database_rejects_
          leaves_stock_untouched
    assert stock("MUG") == 5
E   AssertionError: assert 3 == 5
1 failed, 4 passed
```

Service + real DB: two mugs gone, no order to account for them.

```
SHOP_SERVICE=buggy pytest tests/test_api.py
```

```
5 passed
```

End-to-end: the API's validation blocks the missing email first.

Higher-level tests are not a superset of lower-level ones. Test each integration point where it lives.

Try it: predict, then run

TRY IT 1
students · 7 min

For each cell: will the file pass or fail? Write your prediction first, then run it.

Test file	SHOP_PAYMENTS=v1	SHOP_SERVICE=buggy
`test_order_service_unit.py`	?	?
`test_payments_stubbed.py`	?	?
`test_payments_live.py`	?	?
`test_order_service.py`	?	?
`test_api.py`	?	?

```
# for example
SHOP_PAYMENTS=v1 \
  pytest tests/test_api.py

SHOP_SERVICE=buggy \
  pytest tests/test_api.py
```

Done early? Pick a cell where your prediction was wrong. What did you misjudge about that test?

Read the table by column

LIVE DEMO 4
check the predictions

Test file	v1 client	buggy service	What it shows
`test_order_service_unit.py`	3 passed	3 passed	mocks cannot see either defect
`test_payments_stubbed.py`	**1 failed**	5 passed	the loose stub passes; the contract check fails
`test_payments_live.py`	**3 failed**	6 passed	PayFast answers 400: integer number of cents
`test_order_service.py`	**2 failed**	**1 failed**	v1: charge rejected; buggy: assert 3 == 5
`test_api.py`	**2 failed**	5 passed	assert 502 == 201: but where?

v1 client column

Caught from the contract check upward, but only the lower levels say where.

Buggy service column

Caught at exactly one level: the service, against a real database.

External services are different

1

Down, slow, or limited

Maintenance windows, outages, latency spikes, rate limits.

2

Real effects

Calling it can charge a card, send an email, or cost money.

3

Changes without you

A new API version changes behavior while your code stays the same.

4

Hard to make fail

You cannot ask PayFast for a 503 when you need one.

So the shop's tests replace PayFast with a fake that enforces PayFast's published contract, runs on a real local port, and fails on demand.

The payment client: every line answers a finding

```
payload = {"amount": amount_cents, "currency": "USD",
           "reference": reference}
headers = {"Authorization": f"Bearer {self.api_key}",
           "Idempotency-Key": reference}
for _ in range(self.retries + 1):
    try:
        response = self.session.post(url, json=payload,
            headers=headers, timeout=self.timeout)
    except (requests.ConnectionError, requests.Timeout):
        continue                        # retry
    if response.status_code in RETRYABLE:    # 5xx
        continue                        # retry; don't parse
    break
else:
    raise PaymentUnavailable(problem)
if response.status_code == 201: return response.json()["id"]
if response.status_code == 402: raise PaymentDeclined(...)
raise PaymentError(...)                 # 400, 401: our bug
```

R1 integer cents in amount

R6 Idempotency-Key makes the retry safe

R7 a timeout on every call

R5 5xx bodies are never parsed as JSON

R8 400/401 are ours: no retry

A fake provider, over real HTTP

tests/conftest.py

```
@pytest.fixture
def provider():
    app = create_provider(secret=PROVIDER_SECRET)
    server = make_server("127.0.0.1", 0, app,
                         threaded=True)
    thread = threading.Thread(
        target=server.serve_forever,
        kwargs={"poll_interval": 0.01}, daemon=True)
    thread.start()
    app.url = f"http://127.0.0.1:{server.port}"
    yield app
    server.shutdown()
    thread.join()
```

The real PaymentClient talks to it through the real requests library.

Enforces the contract

Wrong key: 401. Amount not integer cents: 400. A repeated Idempotency-Key is not charged twice.

Fails on demand

provider.config["MODE"] = "decline" or "unavailable" (a 503 with an HTML body); provider.config["DELAY"] for slow answers.

Port 0

The OS picks a free port, so parallel test runs never collide.

Outages, timeouts, wrong keys

LIVE DEMO 5
6 passed, 0.6 s

```
pytest tests/test_payments_live.py -v
```

```
test_charge_succeeds_in_integer_cents          PASSED
test_declined_card                             PASSED
test_wrong_api_key_is_our_error_and_not_...    PASSED
test_outage_is_retried_then_reported_as_...    PASSED
test_slow_provider_times_out                   PASSED
test_same_reference_is_charged_only_once       PASSED
6 passed
```

```
def test_slow_provider_times_out(provider):
    provider.config["DELAY"] = 0.5
    impatient = PaymentClient(
        provider.url, "sk_test_123",
        timeout=0.2)
    with pytest.raises(PaymentUnavailable,
                       match="ReadTimeout"):
        impatient.charge(1250,
                         reference="order-1")
```

A timeout is not a failure; it is an unknown. After the client gives up, the provider may still charge the card a moment later. Production systems reconcile by asking the provider about the Idempotency-Key.

Compensation: undo your own side

shop/service.py

```
try:
    charge_id = self.payments.charge(
        total_cents, reference=f"order-{order_id}")
except PaymentError:
    with self.connection:
        self.repository.release_stock(sku, quantity)
        self.repository.update_order(
            order_id, "payment_failed")
    raise
```

What the tests check

Provider	After place_order
`decline`	PaymentDeclined; stock 5; payment_failed
`unavailable`	PaymentUnavailable; stock 5; payment_failed
`normal`	paid; stock 3; PayFast charged 2500 = order total (R2)

The charge happens in someone else's system, so it cannot join your transaction. When it fails, undo your own side by hand, and test that you did, on both sides.

Where to draw the boundary

Test file	API	Service	Database	PayFast
`test_order_service_unit.py`	-	real	mock	mock
`test_repository.py`	-	-	**real**	-
`test_payments_stubbed.py`	-	-	-	stub (responses)
`test_payments_live.py`	-	-	-	**fake, real HTTP**
`test_order_service.py`	-	real	**real**	**fake, real HTTP**
`test_api_topdown.py`	**real**	stub	-	-
`test_api.py`	**real**	real	**real**	**fake, real HTTP**

Make real what you are testing; double what lies beyond it. Especially anything slow, costly, or with real-world effects.

The stub that agrees with the bug

LIVE DEMO 6
loose stub vs. contract

Loose stub: passes for the v1 client

```
responses.post(CHARGES, status=201,
    json={"id": "ch_1", "status": "succeeded"})
assert client.charge(1250,
                     reference="order-1") == "ch_1"
```

Contract check: the request itself

```
sent = responses.calls[0].request
assert json.loads(sent.body) == {
    "amount": 1250, "currency": "USD",
    "reference": "order-1"}
assert sent.headers["Idempotency-Key"] == "order-1"
```

```
SHOP_PAYMENTS=v1 pytest tests/test_payments_stubbed.py --tb=short
```

```
E     Differing items:
E     {'amount': '12.50'} != {'amount': 1250}
1 failed, 4 passed
```

Write doubles from the other side's specification, not from your own code. Check requests as well as responses.

Fakes and contract tests

1

Fake

A working implementation that takes a shortcut unsuitable for production.

Here: fakes/payment_provider.py (about 60 lines of Flask); the main suite's RecordingEmailClient, which records messages instead of sending them.

2

Contract test

Checks that a double still behaves like the real thing.

Here: The consumer records the requests it sends and the answers it expects; the provider's own suite replays them against the real provider (e.g. Pact).

Keep at least one test against something that enforces the contract.

Try it: unexpected dependency behavior

TRY IT 2
students · 10 min

1. PayFast answers 201 Created, but the body has no id. What does charge() do today? Write a responses test to find out.

```
@responses.activate
def test_201_without_an_id(client):
    responses.post(CHARGES, status=201,
                   json={"status": "pending"})
    client.charge(1250, reference="order-1")
    # what happened?
```

2. What should it do instead? Change the client, then make your test assert that behavior.

```
with pytest.raises(...):
    client.charge(1250, reference="order-1")
```

Done early? Add the same case to the fake provider, and a test against it over real HTTP.

Back in the main suite

LIVE DEMO 7
tests/integration/

```
cd ../..          # demo-testing-suite/
pytest -m integration -rx
```

```
...registered_user_is_committed_to_the_database  PASSED
...duplicate_name_is_rejected_and_not_emailed..  PASSED
...database_enforces_unique_names_even_without.. PASSED
...failed_welcome_email_does_not_leave_a_half_
   registered_user                                XFAIL
3 passed, 75 deselected, 1 xfailed
```

The schema lived only in a test

The users table was created by a test fixture since Week 2. Now app/ has create_schema(), with NOT NULL UNIQUE names.

A half-registered user

If the email fails, register() has already saved the user: never welcomed, and a retry is rejected as a duplicate.

Documented, not fixed: `@pytest.mark.xfail(strict=True, reason="KNOWN ISSUE (Week 9): ...")`. When a fix lands, strict=True makes the test fail (XPASS(strict)) until the marker is removed.

Organizing integration tests

Separate

tests/integration/ next to the unit-test folders: anyone can see which tests assemble real parts.

Mark

pytestmark = pytest.mark.integration, registered under markers in pytest.ini.

Assemble in fixtures

conftest.py builds database, fake provider, service, and API client; tests read like scenarios.

Isolate

A fresh database file and a fresh provider on a fresh port, per test. No shared state.

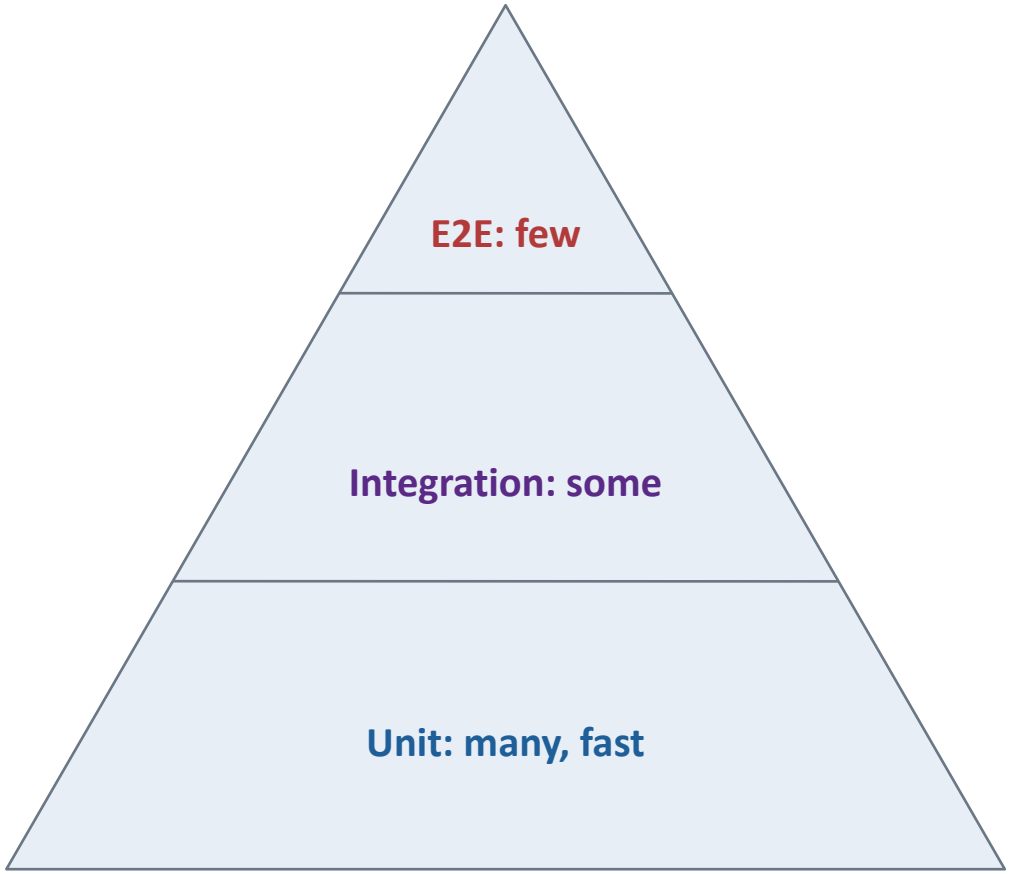
Run either kind on its own

```
pytest -m integration
pytest -m "not integration"
```

```
3 passed, 75 deselected,
  1 xfailed
1 failed, 72 passed,
  4 deselected, 2 xfailed
```

The second line is exactly Week 8's result: the one failure and two xfails are the Week 1 and Week 3 bugs.

How many, and where they run



slower and costlier toward the top

.github/workflows/tests.yml (sketch)

```
jobs:
  unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest -m "not integration"
  integration:
    needs: unit     # only if unit tests pass
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest -m integration
```

Common integration failures

Failure	Example this week	Found by
Incompatible interfaces	service exceptions vs. API status codes	top-down API tests with a stub (R9)
Incorrect data formats	"12.50" dollars vs. 1250 cents	request contract check; fake provider (R1)
Database inconsistencies	stock reserved, order never created	service + real DB, via a second connection (R3)
Service failures	provider down; HTML 503 body	fake provider in "unavailable" mode (R5)
Authentication problems	wrong provider key answered as "retry later"	wrong-key tests at client and API (R8)
Unexpected behavior	slow responses; SQLite ignoring foreign keys	slow fake; foreign-key test (R7, R10)
Partial failure	user saved, welcome email failed	main-suite integration test (xfail)

Try it: fix the half-registered user

TRY IT 3
students · 10 min

1. Choose a design: delete the user if the email fails, or save with a 'welcome pending' flag and retry later.

```
app/accounts/registration_service.py

def register(self, name, email): ...
```

2. Implement it and run the integration tests. Read the result, then remove the xfail marker.

```
pytest -m integration
# XPASS(strict): the fix landed;
# now remove the marker and rerun
```

Done early? Which design is easier to test, and why?

Practice question

TRY IT
think, then vote · 2 min

PaymentClient times out (0.2 s per attempt, one retry) and raises PaymentUnavailable.

What do we actually know about the charge?

A. It failed, so releasing the stock is always safe.

B. Its outcome is unknown: the provider may still charge; reconcile by Idempotency-Key.

C. Retrying with a new Idempotency-Key is safe.

D. The provider is down, so every order should get 503 for 30 seconds.

Answer: B

LIVE DEMO 8
optional

B **The outcome is unknown: the provider may still charge; reconcile by Idempotency-Key.**

+	**Why**	The client gave up after two 0.2 s attempts with no charge yet; a second later the fake had charged the card, once.
×	**A**	Releasing stock for an order that was actually paid leaves a charge with no order.
×	**C**	A new key is a new charge: that is exactly the double charge the key prevents.
×	**D**	One slow response is not an outage; the client already retried.

Takeaways, and next week

1

Real database, second connection

Constraints and transactions only exist in the real thing; check what was committed, not what you wrote.

2

Fakes that enforce, stubs that check

Write doubles from the other side's spec, check requests too, and test the failures.

3

Separate, mark, isolate

tests/integration/, an integration marker, a database and port per test, and CI.

Next week: System Testing. The shop already runs as a whole system over HTTP (python serve.py).

Homework: Exercises 3-5 in the reading: refunds, the half-registered user, and a 201 without an id.